

110.1. La fonction poserPanneau() — une tuile dans le panneau

#include <poserPanneau> : `poserPanneau(colonne, ligne, tuile)` pose une tuile dans le panneau.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `poserPanneau(colonne, ligne, tuile)` pose une tuile **dans le panneau** : un cœur par vie, une icône.

La ligne à écrire : `#include <poserPanneau>`.

Ses arguments : comme `poser()`.

Ce qu'elle coûte : environ **120 octets** la première fois.

Essaie : pose trois cœurs avec une boucle `for`.

On retrouve poserPanneau dans 1 leçon

[110.3](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **190 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         28 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         16 o ← CopierTuiles – 1 lettre sur 43
```

Lettres gravées : B

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
panneau()      x 1   15 o sur place, dans main()
poserPanneau() x 1    6 o sur place, dans main()
image()        x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

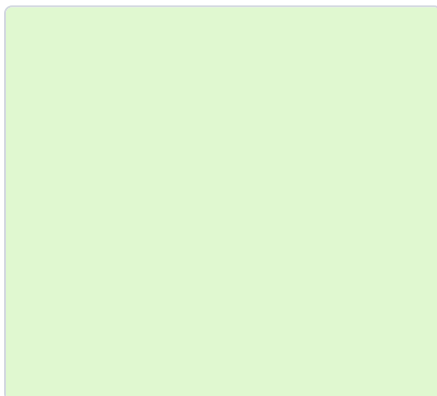
```
// ---- #include <poserPanneau> : une tuile dans le panneau ----  
  
#include <poserPanneau>    // poserPanneau() : une tuile dans le panneau  
#include <ALPHABET>  
#include <panneau>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

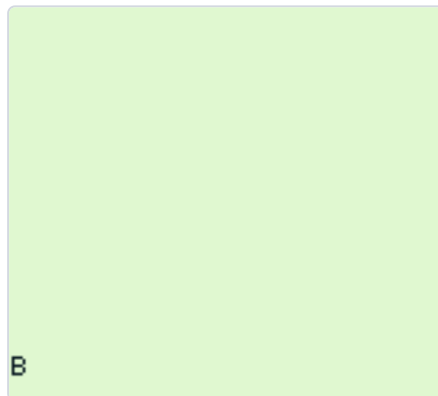
```
int main() {  
    poserPanneau(0, 0, ALPHABET[1]);    // un B dans la première case du panneau  
    panneau(0, 128);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un B tout à gauche de la bande du bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B (tuile 2) est dans la case (0, 0) du panneau (2)

Ensuite : **110.2. La fonction lirePanneau() — lire une case du panneau** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++